

diffusion-tinker: reproduction results

Srijit Iyer

July 2026

diffusion-tinker runs RL post-training for diffusion models: give it a model, a reward, and prompts, and it runs one of six algorithms (FlowGRPO, DDRL, DRaFT, DiffusionDPO, DDPO, SFT) behind one `Trainer/Config` API on top of `diffusers`. Targets SD3.5 and FLUX.1.

Every number below is a real run on the Stanford cluster (A5000, or A6000-Ada for FLUX), LoRA rank 16, bf16. Code was fixed first (Appendix A).

1 When RL actually helps

The reward goes up when the base model has room to improve, and stays flat when it is already good at the reward. OCR, counting, and CLIP-score climb. PickScore and HPS don't, because SD3.5 is already near their ceiling. So PickScore wasn't broken earlier, it was just the wrong reward to show a gain on.

2 Results

Trainer	Result	Status
FlowGRPO	OCR 1.00, count 0.79→1.0, CLIP 27→29	validated
DRaFT	aesthetic 5.9 → 9.6	validated
DiffusionDPO	winner/loser separation learned	validated
SFT	Naruto denoising loss converges	validated
DDRL	climbs with anchor, collapses without (§3)	needs anchor
DDPO/DPOK	ratio ~1.0 but reward won't climb	weak

DDPO is the one trainer that won't reliably move a reward, even on an easy task (counting dropped 0.34→0.10, OCR flat). That's the algorithm, not a bug: the ratio is fixed and gradients reach the weights. It's just PPO without GRPO's group-relative advantage.

3 The grid: algorithm × anchor × reward

FlowGRPO vs DDRL, three anchor strengths each, on counting and OCR. Same budget everywhere (group 12, 20 steps, noise 0.1, lr 10^{-4}).

The anchor story holds directionally (no anchor collapses, an anchor buys stability and a climb), but $\beta = 0.3$ delays the collapse rather than stopping it.

4 Counting: a reward that climbs

FlowGRPO on an object-counting reward (right number of objects, checked with an open-vocab detector). Count accuracy goes from ~0.79 to 1.0 and holds near 0.9.

FlowGRPO vs DDRL across anchor strength on two rewards
FlowGRPO's KL is robust (climbs/stable); DDRL collapses without a strong anchor - even $\beta=0.3$ only delays it

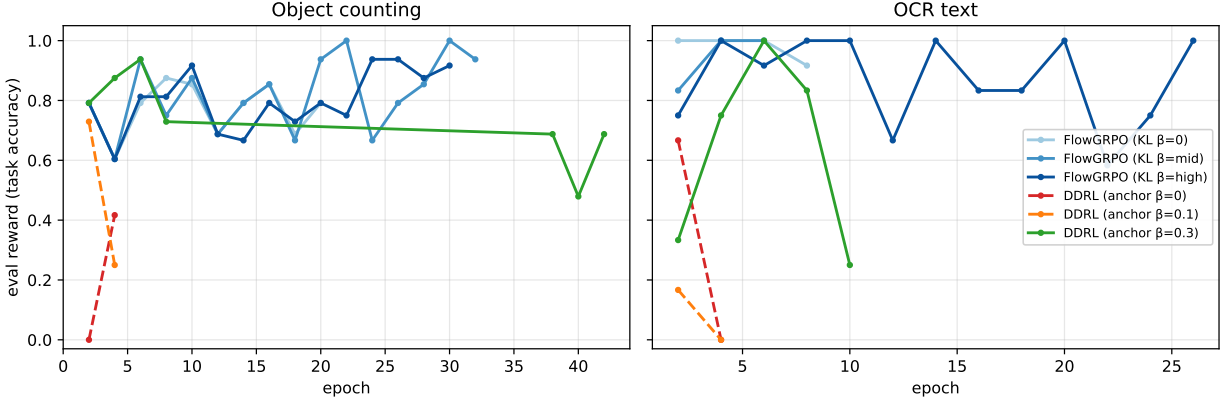


Figure 1: FlowGRPO's KL holds high at every β on both rewards and never collapses. DDRL collapses with no anchor ($\beta = 0$, red), as expected. A bigger anchor ($\beta = 0.3$, green) climbs first but only delays it, on OCR it hits 1.0 then crashes to 0.25. So the data anchor works but is weaker than the KL, and needs a larger β to hold the whole run.

5 FLUX

The FLUX path runs end-to-end (FlowGRPO + aesthetic, 48 GB card). It's noisy: schnell is a 4-step distilled model, bad for the stochastic sampling these methods need, and its ratio sits near 0.92 vs ~ 1.0 on SD3. Code path works; FLUX.1-dev would be the better target and hasn't been run.

6 What's not done

Eval uses few prompts, so curves are noisy, read the trend. DDPO doesn't reliably help. DDRL's anchor at $\beta = 0.3$ delays but doesn't stop collapse. DiffusionDPO's re-run is blocked by Pick-a-Pic being gated on HF. FLUX.1-dev unrunk.

A Verification

The library passed its tests and smoke runs, but several trainers weren't learning, a smoke run that makes an image and a reward tells you nothing about whether gradients reach the weights or the reward moves. What that turned up:

- **One optimizer step per epoch.** The RL trainers folded a whole batch into one update, so a long run did a few dozen steps total. Stepping per mini-batch is what let the reward move.
- **Ratio bias.** Training the last near-zero-noise step pushed the sampling-vs-replay ratio to ~ 0.92 ; dropping it fixed that. Sampling also ran without autocast while replay ran with it, which I aligned.
- **DRaFT never trained.** A no-grad warmup cached detached bf16 casts of the LoRA weights, which the grad loop reused, so LoRA gradients came back `None`. Fixed with `cache_enabled=False` (reward 5.9 $\rightarrow$ 9.6).

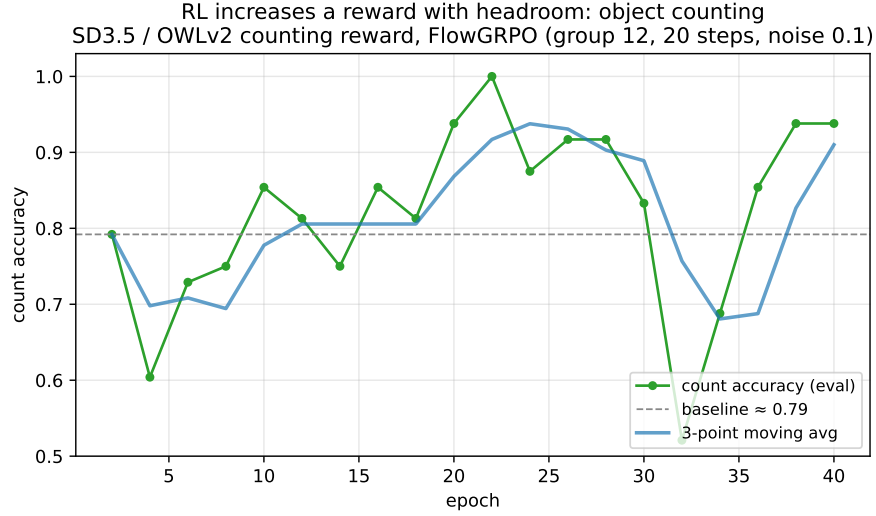


Figure 2: One catch worth knowing: a detector reward needs coherent images, so it wants more sampling steps and less noise than PickScore. Too few steps and the detector finds nothing, so the reward sits at zero and nothing trains.

- **VAE range, both ways.** Decode clamped $[-1, 1]$ to $[0, 1]$ with no $/2 + 0.5$ (55% of every image went black); encode fed $[0, 1]$ to a VAE wanting $[-1, 1]$.
- **Memory.** The old trajectory stayed alive into the next epoch’s sampling (2x peak memory) and the VAE decoded a whole group at once, freed per epoch and chunked the decode. Plus smaller fixes to the PEFT API, reward APIs across `transformers` versions, and the noise/KL interaction (low noise blows up the KL penalty as $1/\text{noise}^2$ and freezes the policy).

The earlier “validated” numbers were on broken images and latents, so everything here is a fresh run on fixed code. Code and logs: <https://github.com/srijitier/diffusion-tinker>.